Binary Tree Node Counting Using C++

Complete project documentation

Contents

- 1. Problem Statement
- 2. Objective
- 3. Introduction
- 4. Types of Nodes in a Binary Tree
- 5. Algorithm
- 6. Flowchart
- 7. Program (C++ Source Code)
- 8. Explanation of the Code
- 9. Working of the Program
- 10. Output
- 11. Test Cases
- 12. Time and Space Complexity
- 13. Advantages and Limitations
- 14. Applications
- 15. Possible Extensions
- 16. Viva Questions and Answers
- 17. Conclusion
- 18. References

1. Problem Statement

Write a C++ program to create a binary tree and count the total number of nodes, leaf nodes, internal nodes, and nodes having exactly one child. The program should traverse the binary tree using recursion, identify each type of node, and display the final count of all four categories.

Item	Details
Input	A binary tree (built in the program by linking nodes through left and right pointers).
Output	Total nodes, leaf nodes, internal nodes and nodes having exactly one child.
Method	Recursive preorder traversal with four counters.
Language	C++ (standard library only: iostream).
Constraint	Each node must be visited exactly once; the tree may be empty.

2. Objective

The main objective of this program is to understand the structure of a binary tree and identify different types of nodes. It aims to count the total number of nodes, leaf nodes, internal nodes, and nodes having exactly one child using C++ programming. It also helps in understanding recursive functions, pointers, and tree traversal techniques.

- To create a binary tree using C++ structures, pointers and dynamic memory (`new`).
- To count the total number of nodes.
- To identify and count leaf nodes.
- To identify and count internal nodes.
- To count nodes having exactly one child.
- To understand recursive tree traversal, base case and recursive case.
- To pass variables by reference so that all recursive calls update the same counters.
- To analyse the time and space complexity of a recursive algorithm.

3. Introduction

3.1 Tree data structure

A tree is a non-linear data structure used to represent hierarchical relationships such as a family tree, a company chart or the folders on a computer. Unlike arrays, linked lists, stacks and queues, which store data in a line, a tree branches out from a single starting node.

3.2 Binary tree

A binary tree is a non-linear data structure in which each node can have a maximum of two children, known as the left child and the right child. The first node of the tree is called the root node. Each node contains data and pointers that connect it to its children. Binary trees are widely used in computer science for searching, sorting, expression evaluation, and hierarchical data representation.

```
struct Node {
    int data;
    Node *left;
    Node *right;
};
```

3.3 Terminology

Term	Meaning
Root	The topmost node; it has no parent.
Parent / Child	A node and the nodes directly connected below it.
Siblings	Nodes that share the same parent.
Leaf node	A node with no children (both pointers are NULL).
Internal node	A node with at least one child.
Edge	The link between a parent and a child.
Level	Distance of a node from the root (root is at level 0).
Height	Number of edges on the longest path from the root to a leaf.
Subtree	A node together with all of its descendants.

Term	Meaning
Degree of a node	Number of children of the node (0, 1 or 2 in a binary tree).

3.4 Common kinds of binary trees

Kind	Description
Full binary tree	Every node has either 0 or 2 children, so no node has exactly one child.
Complete binary tree	All levels are completely filled except possibly the last, which is filled from the left.
Perfect binary tree	All leaves are at the same level and every internal node has two children.
Skewed binary tree	Every node has only one child, so the tree looks like a linked list.
Balanced binary tree	The heights of the left and right subtrees of every node differ by at most 1.

3.5 About this program

In the Binary Tree Node Counting program, a binary tree is created using C++. The program uses a recursive function to visit each node and count different categories of nodes. It checks whether a node has zero, one, or two children and updates the corresponding counters. Finally, the program displays the total number of nodes, leaf nodes, internal nodes, and nodes having exactly one child.

4. Types of Nodes in a Binary Tree

A binary tree contains different types of nodes based on the number of children connected to each node.

Type	Definition	Condition in code
Total nodes	All the nodes present in the tree, including the root, internal nodes and leaf nodes.	`root != NULL` (counted for every real node)
Leaf nodes	Nodes that do not have any children. Both child pointers are NULL.	`left == NULL && right == NULL`
Internal nodes	Nodes that have at least one child, either one or two. The root is internal if it has at least one child.	not a leaf
Exactly one child	Nodes with only a left child or only a right child, never both.	`left == NULL || right == NULL` (when not a leaf)

Important points

- Every node is counted only once in the total node count.
- A leaf node has zero children.
- An internal node has at least one child.
- A node with exactly one child is also an internal node.
- A node with two children is an internal node but is not a one-child node.

Relations between the counts

Relation	Explanation
total = leaf + internal	Every node is either a leaf or an internal node.
internal = oneChild + twoChild	Internal nodes have one or two children.
leaf = twoChild + 1	True for every non-empty binary tree; useful to verify the output.

Proof of the last relation. A tree with n nodes has n - 1 edges. Counting edges from the parents' side gives n1 + 2 x n2, where n1 is the number of one-child nodes and n2 the number of two-child nodes. So n - 1 = n1 + 2 x n2. Since n = n0 + n1 + n2 (n0 = leaves), we get n0 + n1 + n2 - 1 = n1 + 2 x n2, which simplifies to n0 = n2 + 1.

5. Algorithm

The algorithm starts by defining a node structure and creating the tree. Four counters (total, leaf, internal and one-child) are set to zero and a recursive function is called with the root.

1. Start the program and define the node structure containing data, a left pointer and a right pointer.
2. Create the binary tree by allocating nodes and connecting them through their left and right pointers.
3. Initialize four counters (total, leaf, internal, oneChild) to zero.
4. Call the recursive function by passing the root node.
5. If the current node is NULL, return from the function.
6. Otherwise, increase the total node counter by one.
7. If both child pointers are NULL, increase the leaf node counter.
8. If at least one child exists, increase the internal node counter; if exactly one child exists, also increase the one-child counter.
9. Recursively visit the left subtree and then the right subtree.
10. Repeat until all nodes have been visited, then display the four counters and stop.

Pseudocode

```
CountNodes(node)
    if node is NULL then return
    total <- total + 1
    if node.left is NULL and node.right is NULL then
        leaf <- leaf + 1
    else
        internal <- internal + 1
        if node.left is NULL or node.right is NULL then
            oneChild <- oneChild + 1
    CountNodes(node.left)
    CountNodes(node.right)
```

6. Flowchart

The flowchart shows the sequence of operations of the recursive function. It checks whether the current node is NULL; if not, the node is counted and its children are examined to decide whether it is a leaf node, an internal node, or a node having exactly one child. The function then repeats the process for the left and right subtrees. (Detailed flowcharts are given in the separate Flowchart document.)

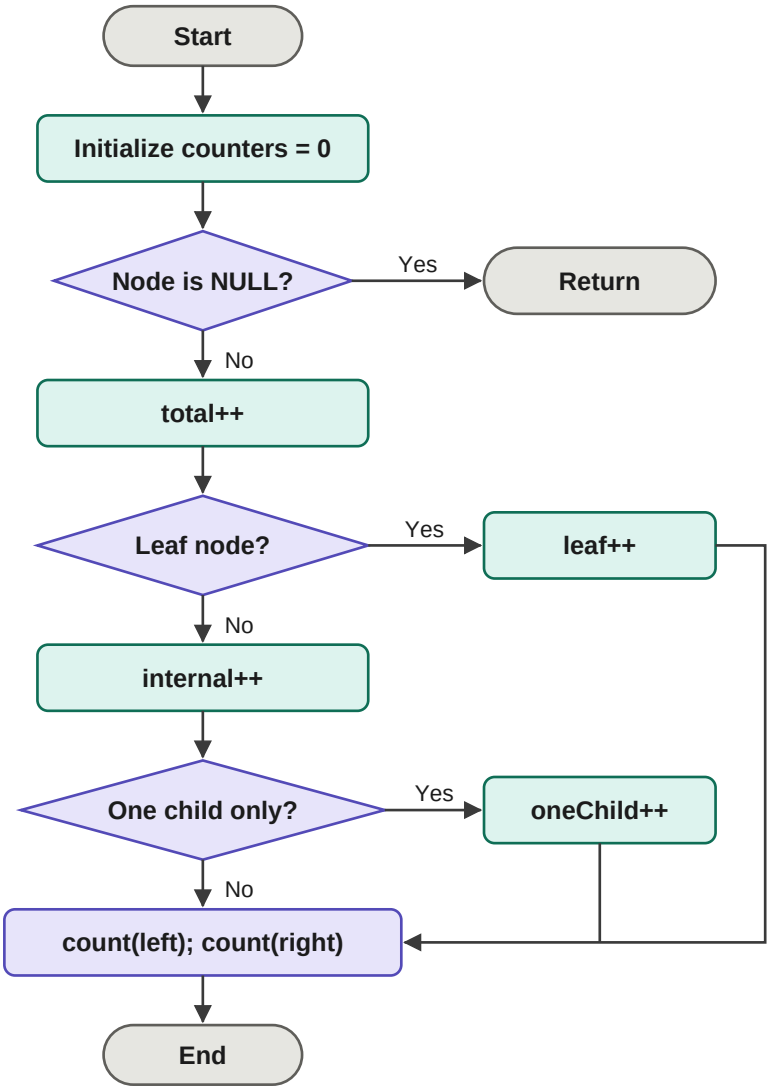

Figure 1: Flowchart of the recursive function countNodes(node)

7. Program (C++ Source Code)

```
#include <iostream>
using namespace std;

// Node structure of the binary tree
struct Node {
    int data;
    Node *left;
    Node *right;
};

// Create and return a new node
Node* createNode(int value) {
    Node* newNode = new Node;
    newNode->data  = value;
    newNode->left  = NULL;
    newNode->right = NULL;
    return newNode;
}

// Recursive function: counts the four node categories
void countNodes(Node* root, int &total, int &leaf,
                int &internal, int &oneChild) {
    if (root == NULL)                // base case
```

```
        return;

    total++;                        // every real node is counted

    if (root->left == NULL && root->right == NULL) {
        leaf++;                     // no children
    } else {
        internal++;                 // at least one child
        if (root->left == NULL || root->right == NULL)
            oneChild++;             // exactly one child
    }

    countNodes(root->left,  total, leaf, internal, oneChild);
    countNodes(root->right, total, leaf, internal, oneChild);
}

// Free the memory used by the tree (postorder)
void deleteTree(Node* root) {
    if (root == NULL) return;
    deleteTree(root->left);
    deleteTree(root->right);
    delete root;
}

int main() {
    // Build the sample tree
    Node* root = createNode(1);
    root->left               = createNode(2);
    root->right              = createNode(3);
    root->left->left         = createNode(4);
    root->right->left        = createNode(5);
    root->right->right       = createNode(6);
    root->right->left->left  = createNode(7);

    int total = 0, leaf = 0, internal = 0, oneChild = 0;
    countNodes(root, total, leaf, internal, oneChild);

    cout << "Total nodes          : " << total    << endl;
    cout << "Leaf nodes           : " << leaf     << endl;
    cout << "Internal nodes       : " << internal << endl;
    cout << "Nodes with one child : " << oneChild << endl;

    deleteTree(root);
    return 0;
}
```

8. Explanation of the Code

Part of the code	Explanation
`#include <iostream>`	Header needed for cout and endl (console output).
`struct Node`	Defines a tree node: an integer data field and two Node pointers, left and right.
`createNode(int value)`	Allocates a new node with new, stores the value and sets both child pointers to NULL so that the new node is a leaf until it is linked.
`countNodes(...)`	The recursive function. The counters are passed by reference (&) so that every call updates the same variables.
`if (root == NULL) return;`	Base case. Stops the recursion when an empty subtree is reached.
`total++`	Counts the current node. Executed once per real node.
`left == NULL && right == NULL`	Leaf test. If true the leaf counter is increased.
`else { internal++; ... }`	At least one child exists, so the node is internal. The inner if checks for exactly one NULL pointer and increments oneChild.

Part of the code	Explanation
`countNodes(root->left, ...)`	Recursive call for the left subtree.
`countNodes(root->right, ...)`	Recursive call for the right subtree.
`deleteTree(root)`	Releases all allocated memory using a postorder traversal (children are deleted before the parent).
`main()`	Builds the sample tree manually, initializes the counters, calls countNodes and prints the four results.

9. Working of the Program

The program creates a binary tree in which each node stores a value and contains pointers to its left and right children. The recursive function starts from the root node and visits every node in the tree. At each node, it checks the child pointers and increases the appropriate counters. If both child pointers are NULL, the node is classified as a leaf node. If at least one child exists, it is classified as an internal node. If exactly one child exists, the one-child counter is increased. The function continues until every node has been processed.

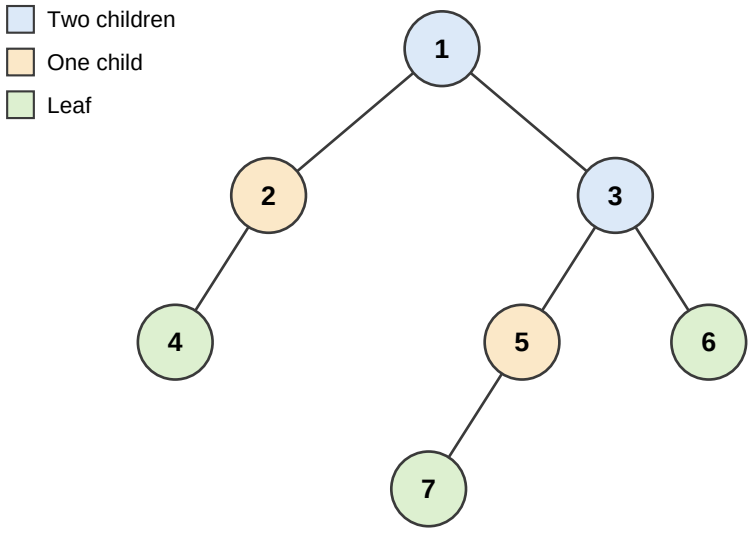

Figure 2: Example binary tree (colour shows the type of each node)

Example tree used in all documents: node 1 is the root with children 2 and 3; node 2 has only a left child (4); node 3 has children 5 and 6; node 5 has only a left child (7).

Step	Node visited	Left child	Right child	Type	total	leaf	internal	oneChild
1	1	2	3	Two children	1	0	1	0
2	2	4	NULL	One child	2	0	2	1
3	4	NULL	NULL	Leaf	3	1	2	1
4	3	5	6	Two children	4	1	3	1
5	5	7	NULL	One child	5	1	4	2
6	7	NULL	NULL	Leaf	6	2	4	2
7	6	NULL	NULL	Leaf	7	3	4	2

The nodes are visited in preorder: 1, 2, 4, 3, 5, 7, 6. The recursion goes down to the left as far as possible, returns at each NULL pointer, and then continues with the right subtrees.

10. Output

For the example tree, the program prints:

```
Total nodes          : 7
Leaf nodes           : 3
Internal nodes       : 4
Nodes with one child : 2
```

Category	Count	Nodes
Total nodes	7	1, 2, 3, 4, 5, 6, 7
Leaf nodes	3	4, 6, 7
Internal nodes	4	1, 2, 3, 5
Nodes having exactly one child	2	2 (left child 4), 5 (left child 7)
Nodes having two children (for checking)	2	1, 3

Check: total (7) = leaf (3) + internal (4), and leaf (3) = two-child nodes (2) + 1. The output was verified by compiling and running the program.

11. Test Cases

Test	Tree	total	leaf	internal	oneChild
1	Empty tree (root = NULL)	0	0	0	0
2	Single node	1	1	0	0
3	Left-skewed tree with 4 nodes	4	1	3	3
4	Right-skewed tree with 4 nodes	4	1	3	3
5	Perfect tree with 7 nodes	7	4	3	0
6	Full tree with 5 nodes (root, two children, left child has two leaves)	5	3	2	0
7	Example tree of this document	7	3	4	2

12. Time and Space Complexity

The time complexity of the program is **O(n)** because every node is visited exactly once during the recursive traversal. The auxiliary space complexity is **O(h)** because the recursive function uses stack space according to the height of the tree, where h represents the tree height and n represents the total number of nodes. In the worst case, a completely skewed tree can require O(n) auxiliary stack space.

Case	Time	Auxiliary space
Balanced tree	O(n)	O(log n)
Skewed tree (worst case)	O(n)	O(n)
Empty tree	O(1)	O(1)

13. Advantages and Limitations

Advantages

- The code is short, clear and follows the natural recursive structure of a tree.
- A single traversal gives all four counts, so no separate passes are needed.
- Linear time, which is the best possible because each node must be examined.
- The results can be verified with simple identities.

Limitations

- Very deep (skewed) trees can cause a stack overflow because of deep recursion; an iterative queue version avoids this.
- The sample program builds a fixed tree; reading the tree from the user needs extra input code.
- Memory must be released manually (deleteTree) when raw pointers are used.

14. Applications

Binary tree node counting is useful for analyzing the structure and size of a tree. It helps identify terminal nodes in decision trees and understand the relationships between parent and child nodes. It is also useful for studying recursive algorithms, tree traversal, searching techniques, and hierarchical data structures.

- Checking whether a tree is a full binary tree (no one-child nodes).
- Finding the size and shape of expression trees and parse trees in compilers.
- Analysing file and folder hierarchies.
- Evaluating balance and structure of binary search trees.
- Counting terminal outcomes (leaves) in decision trees.

15. Possible Extensions

- Build the tree from user input using binary search tree insertion or level-order input.
- Add the count of nodes with two children and display the check leaf = twoChild + 1.
- Find the height and the number of levels of the tree.
- Print the nodes of each category (for example list all leaf values).
- Use the iterative queue version for very tall trees.
- Use smart pointers (std::unique_ptr) to manage memory automatically.

16. Viva Questions and Answers

Q	Question	Answer
1	What is a binary tree?	A tree in which every node has at most two children, called left and right.
2	What is a leaf node?	A node with no children; both left and right pointers are NULL.
3	What is an internal node?	A node with at least one child.
4	Is a node with one child internal?	Yes. It is internal because it has a child, and it is also counted in the one-child count.
5	What is the base case here?	root == NULL; an empty subtree has nothing to count.
6	Why are the counters passed by reference?	So that all recursive calls update the same variables.

Q	Question	Answer
7	Which traversal is used?	Preorder (Root, Left, Right). Any traversal gives the same counts.
8	What is the time complexity?	O(n), as every node is visited once.
9	What is the space complexity?	O(h) for the recursion stack; O(n) in the worst case of a skewed tree.
10	How many leaves does a full binary tree with n internal nodes have?	n + 1 leaves.
11	How can you verify your output?	Check total = leaf + internal and leaf = nodes with two children + 1.
12	How can recursion be avoided?	Use a queue (level-order traversal) or an explicit stack.

17. Conclusion

The Binary Tree Node Counting program demonstrates how to count different types of nodes using C++ and recursive traversal. It identifies the total number of nodes, leaf nodes, internal nodes, and nodes having exactly one child by checking the left and right child pointers. This program provides a practical understanding of binary tree structures, pointers, recursion, and algorithm complexity.

18. References

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms*, MIT Press.
- Mark Allen Weiss, *Data Structures and Algorithm Analysis in C++*, Pearson.
- Ellis Horowitz, Sartaj Sahni, Dinesh Mehta, *Fundamentals of Data Structures in C++*.
- Bjarne Stroustrup, *The C++ Programming Language*, Addison-Wesley.